EX 1: THREE-LAYER NEURAL NETWORK FROM SCRATCH

DATE:

AIM

To design and implement a three-layer feedforward neural network from scratch using Python and NumPy, train it using the backpropagation algorithm with sigmoid activation and cross-entropy loss, and recognize handwritten digits from the MNIST dataset.

PROCEDURE

1. **Import the required libraries, including NumPy, Matplotlib, TensorFlow, and OneHotEncoder from scikit-learn.**
2. **Load the MNIST handwritten digit dataset and preprocess the images by reshaping them into 784-dimensional vectors and normalizing pixel values between 0 and 1.**
3. **Convert the digit labels into one-hot encoded vectors.**
4. **Initialize the weights and biases for the input, hidden, and output layers.**
5. **Define the sigmoid activation function, its derivative, and the cross-entropy loss function.**
6. **Perform forward propagation to calculate the activations of the hidden and output layers.**
7. **Calculate the loss and use backpropagation to compute gradients for the weights and biases.**
8. **Update the weights and biases using gradient descent for 10 epochs.**
9. **Plot the training loss to visualize the learning performance.**
10. **Predict a handwritten digit from the MNIST dataset and display the image with its predicted label.**

CODE

```
import numpy as np

import matplotlib.pyplot as plt

from tensorflow.keras.datasets import mnist

from sklearn.preprocessing import OneHotEncoder


# Load and preprocess the dataset

(X_train, y_train), (_, _) = mnist.load_data()


X_train = X_train.reshape(-1, 784) / 255.0


encoder = OneHotEncoder(sparse_output=False)

y_train = encoder.fit_transform(y_train.reshape(-1, 1))


# Initialize the network
```

```
input_size, hidden_size, output_size = 784, 64, 10

np.random.seed(42)

W1 = np.random.randn(input_size, hidden_size) * 0.01
b1 = np.zeros((1, hidden_size))

W2 = np.random.randn(hidden_size, output_size) * 0.01
b2 = np.zeros((1, output_size))

# Activation and loss functions
def sigmoid(x):
    return 1 / (1 + np.exp(-np.clip(x, -500, 500)))

def sigmoid_deriv(x):
    return x * (1 - x)

def loss_fn(y, y_hat):
    return -np.mean(y * np.log(y_hat + 1e-8))

# Train the model
epochs, lr = 10, 0.1
losses = []

for epoch in range(epochs):
    total_loss = 0

    for i in range(X_train.shape[0]):
        x = X_train[i:i+1]
        y = y_train[i:i+1]

        # Forward propagation
        z1 = x @ W1 + b1
```

```
        a1 = sigmoid(z1)

        z2 = a1 @ W2 + b2
        a2 = sigmoid(z2)

        loss = loss_fn(y, a2)
        total_loss += loss

        # Backpropagation
        dz2 = a2 - y
        dW2 = a1.T @ dz2
        db2 = dz2

        dz1 = (dz2 @ W2.T) * sigmoid_deriv(a1)
        dW1 = x.T @ dz1
        db1 = dz1

        # Update weights and biases
        W2 -= lr * dW2
        b2 -= lr * db2

        W1 -= lr * dW1
        b1 -= lr * db1

    epoch_loss = total_loss / X_train.shape[0]
    losses.append(epoch_loss)

    print(f"Epoch {epoch + 1}, Loss: {epoch_loss:.4f}")

# Visualize training loss
plt.plot(range(1, epochs + 1), losses, marker='o')
plt.title("Training Loss")
plt.xlabel("Epoch")
```

```
plt.ylabel("Loss")
plt.grid(True)
plt.show()

# Predict a sample digit
def predict(img):
    img = img.reshape(1, 784) / 255.0

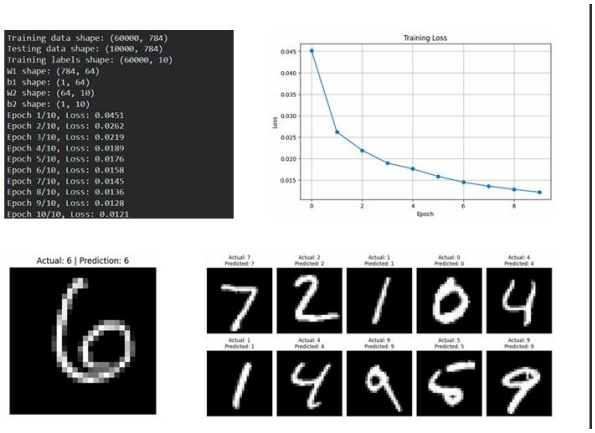
    a1 = sigmoid(img @ W1 + b1)
    a2 = sigmoid(a1 @ W2 + b2)

    return np.argmax(a2)

idx = 100

plt.imshow(X_train[idx].reshape(28, 28), cmap='gray')
plt.title(f"Actual: {encoder.categories_[0][np.argmax(y_train[idx])]}, "
          f"Prediction: {predict(X_train[idx])}")
plt.axis('off')
plt.show()
```

OUTPUT:



RESULT

Thus, a three-layer feedforward neural network was successfully implemented from scratch using Python and NumPy. The network was trained using forward propagation, backpropagation, and gradient descent to recognize handwritten digits from the MNIST dataset. The training loss was visualized, and predictions were performed on sample images.